Chapter 18: Linked Lists

18.1 Introduction to the Linked List ADT

Concept

Dynamically allocated data structures can be linked together in memory to form a chain.

A **linked list** is a sequence of connected data structures, called **nodes**, that are dynamically allocated. This allows a linked list to grow or shrink as needed during program execution. To add data, the program allocates a new node and inserts it into the chain. To remove data, it deletes the corresponding node.

Advantages of Linked Lists Over Arrays and `vector`s

While more complex to manage than arrays, linked lists offer significant advantages:

- **Dynamic Sizing:** A linked list can easily grow or shrink, and its size does not need to be known at compile time.
- **Efficient Insertion and Deletion:** Unlike `vector`s, linked lists excel at inserting or deleting elements. In a `vector`, inserting or removing an element in the middle requires shifting all subsequent elements, which can be time-consuming. In a linked list, these operations only require updating a few pointers, and no other nodes need to be moved.

The Composition of a Linked List

Each node in a linked list contains two main parts:

- One or more members for storing data (e.g., inventory records, customer information).
- A pointer that holds the address of another node.

This structure is illustrated in Figure 18-1.



Figure 18-1 A node

A linked list is formed when each node's pointer points to the next node in the sequence, creating a chain, as shown in Figure 18-2. The list is managed by a pointer called the **list head**, which points to the first node. Each subsequent node points to the next one, and the last node's pointer is set to `nullptr` to mark the end of the list.

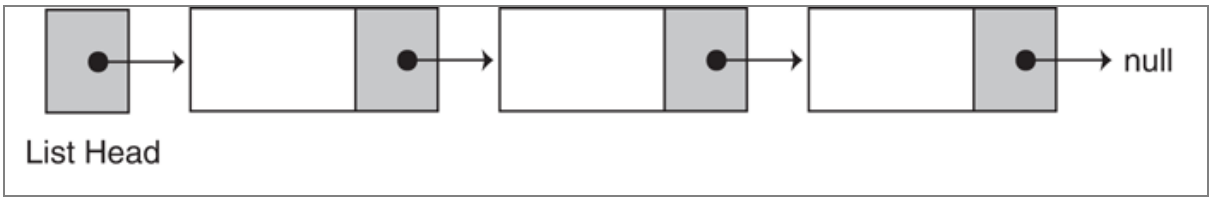


Figure 18-2 Nodes linked by pointers

> **Note**
>
> Figure 18-2 depicts the nodes in the linked list as being neatly arranged in a row. In reality, the nodes may be scattered throughout various parts of memory.

Declarations

To create a linked list in C++, you first need a `struct` to define the node.

```
struct ListNode
{
   double value;
   ListNode *next;
};
```

In this `ListNode` structure:

- `value` is a `double` member that holds the node's data.
- `next` is a pointer that can hold the address of another `ListNode` structure.

Because `ListNode` contains a pointer to an object of its own type, it is a **self-referential data structure**.

Next, a pointer is defined to serve as the list head. An empty list is represented by a head pointer set to `nullptr`.

```
ListNode *head = nullptr;
```

It is crucial to initialize the `head` pointer to `nullptr` before performing any operations on the list.

Checkpoint

18.1 Describe the two parts of a node.

18.2 What is a list head?

18.3 What signifies the end of a linked list?

18.4 What is a self-referential data structure?

18.2 Linked List Operations

Concept

The basic linked list operations are appending a node, traversing the list, inserting a node, deleting a node, and destroying the list.

This section develops a set of functions for linked list operations using the `ListNode` structure. The function declarations are typically stored in a header file, such as `numberlist.h`.

Contents of `numberlist.h`

```
#ifndef NUMBERLIST_H
#define NUMBERLIST_H

struct ListNode
{
   double value;
   ListNode *next;
};

void appendNode(ListNode *&head, double num);
void insertNode(ListNode *&head, double num);
```

```
void deleteNode(ListNode *&head, double num);
void displayList(const ListNode *head);
void destroyList(ListNode *&head);

#endif
```

This procedural implementation includes:

- A set of functions for appending, inserting, and deleting nodes.
- A `displayList` function to show all values in the list.
- A `destroyList` function to deallocate all nodes, preventing memory leaks.

To create an empty list, a `ListNode` pointer is initialized to `nullptr`.

Appending a Node to the List

Appending a Node to a Linked List

Appending a node means adding it to the end of the list. The `appendNode` function takes a reference to the head pointer and a `double` argument, `num`. It creates a new `ListNode`, stores `num` in it, and adds the node to the end of the list.

The general algorithm is as follows:

```
Create a new node.
Store data in the new node.
If there are no nodes in the list
   Make the new node the first node.
Else
   Traverse the list to find the last node.
   Add the new node to the end of the list.
End If.
```

Here is the C++ implementation:

```
void appendNode(ListNode *&head, double num)
{
   ListNode *newNode;
   ListNode *nodePtr;

   newNode = new ListNode;
   newNode->value = num;
   newNode->next = nullptr;
```

```
   if (!head)
      head = newNode;
   else
   {
      nodePtr = head;

      while (nodePtr->next)
         nodePtr = nodePtr->next;

      nodePtr->next = newNode;
   }
}
```

- **Code Breakdown:**
  - `newNode` points to the newly allocated node.
  - `nodePtr` is used to traverse the list.
  - A new node is created, `num` is stored in its `value` member, and its `next` pointer is set to `nullptr` since it will become the last node.
  - If the list is empty (`!head`), the new node becomes the first node by setting `head` to point to it.
  - Otherwise, `nodePtr` starts at the `head` and traverses the list until it reaches the last node (where `nodePtr->next` is `nullptr`).
  - The `next` pointer of the last node is updated to point to `newNode`, attaching it to the end.

Program 18-1 demonstrates this function.

Program 18-1

```
#include <iostream>
#include "numberlist.h"

int main()
{
    ListNode *head = nullptr;

    appendNode(head, 2.5);
    appendNode(head, 7.9);
    appendNode(head, 12.6);

    destroyList(head);
```

```
    return 0;
}
```

(This program displays no output.)

- **Step-by-step execution of Program 18-1:**
  - The `head` pointer is initialized to `nullptr`.
  - **First call:** `appendNode(head, 2.5)`
    - A new node is created with `value` 2.5 and `next` as `nullptr`.
    - Since `head` is `nullptr`, the `if (!head)` condition is true.
    - `head` is updated to point to the new node, making it the first in the list. (Figure 18-3 and Figure 18-4).

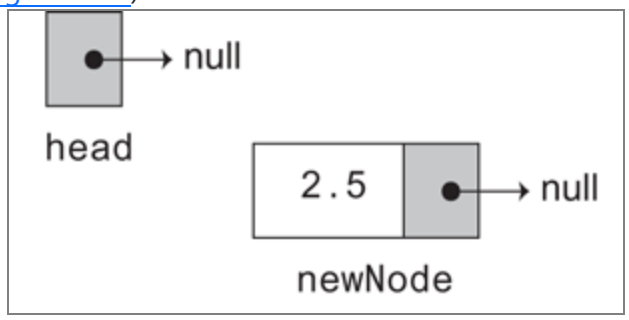


Figure 18-3 State of the `head` pointer and the new node

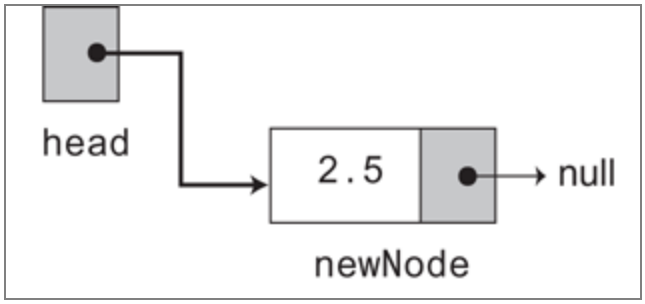


Figure 18-4 `head` points to `newNode`

  - **Second call:** `appendNode(head, 7.9)`
    - A new node is created with `value` 7.9 and `next` as `nullptr` (Figure 18-5).
    - `head` is not `nullptr`, so the `else` block executes.
    - `nodePtr` is initialized to `head` (Figure 18-6).
    - The `while` loop's condition `nodePtr->next` is false, so it does not execute.
    - `nodePtr->next` is set to `newNode`, linking the new node to the end of the list (Figure 18-7).

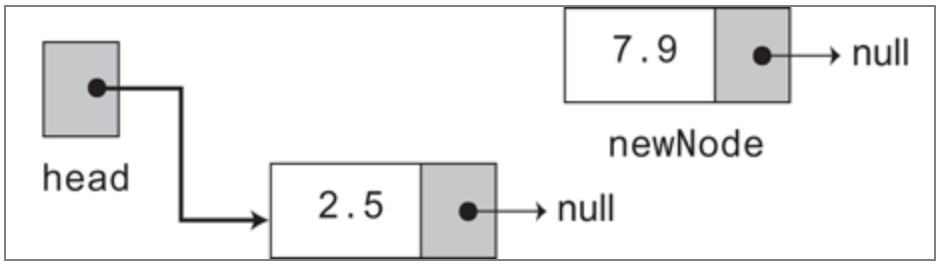


Figure 18-5 A new node is created

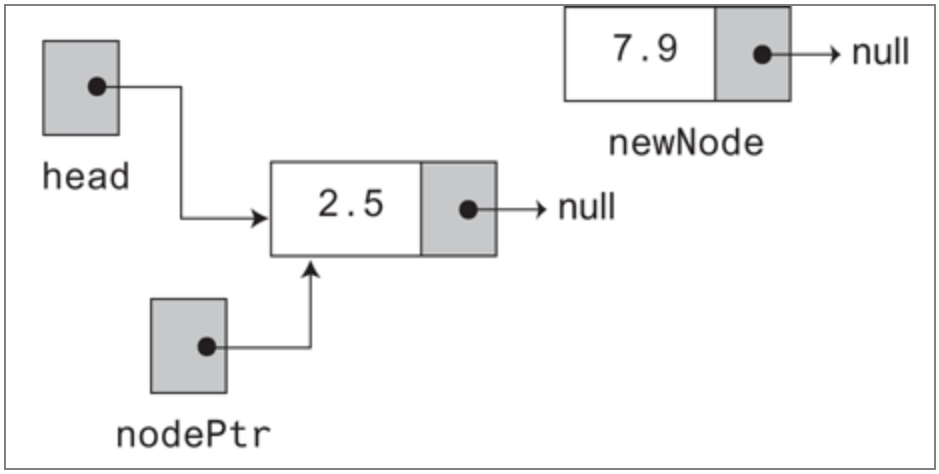


Figure 18-6 `nodePtr` points to the first node

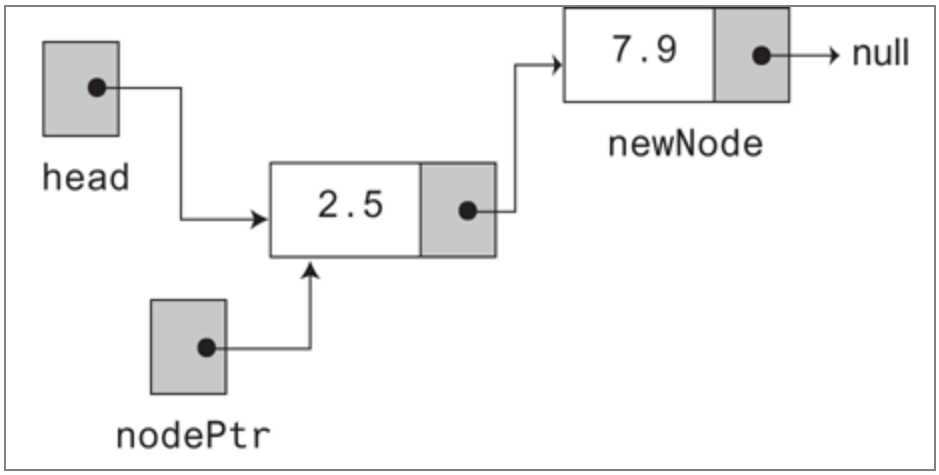


Figure 18-7 The new node is added to the list

- **Third call:** `appendNode(head, 12.6)`
  - A new node is created with `value` 12.6 (Figure 18-8).
  - The `else` block executes, and `nodePtr` is set to `head` (Figure 18-9).
  - The `while` loop executes once, advancing `nodePtr` to the second node (Figure 18-10).

- The loop terminates, and the new node is linked to the end of the list (Figure 18-11).

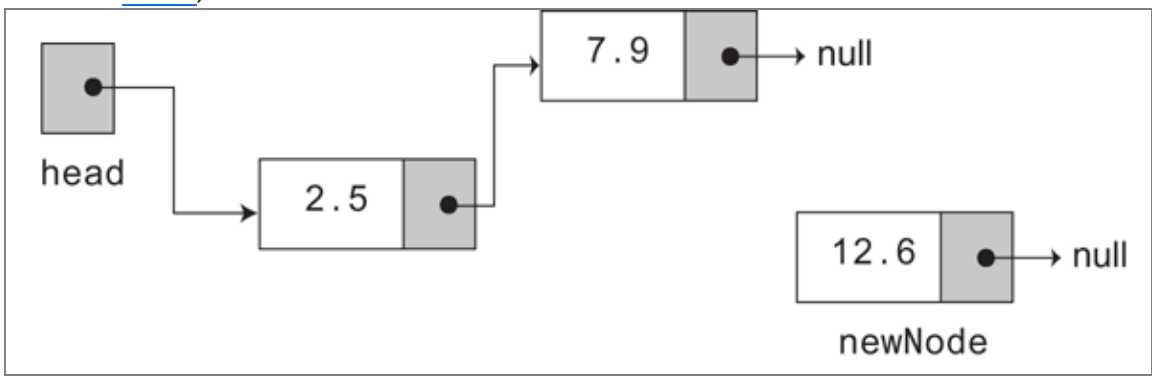


Figure 18-8 A new node is created

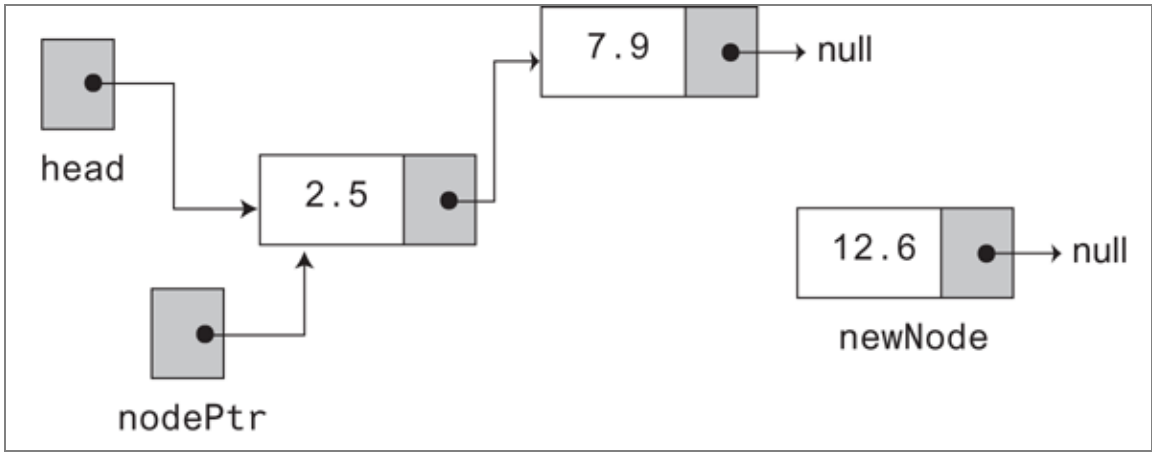


Figure 18-9 `nodePtr` points to the first node

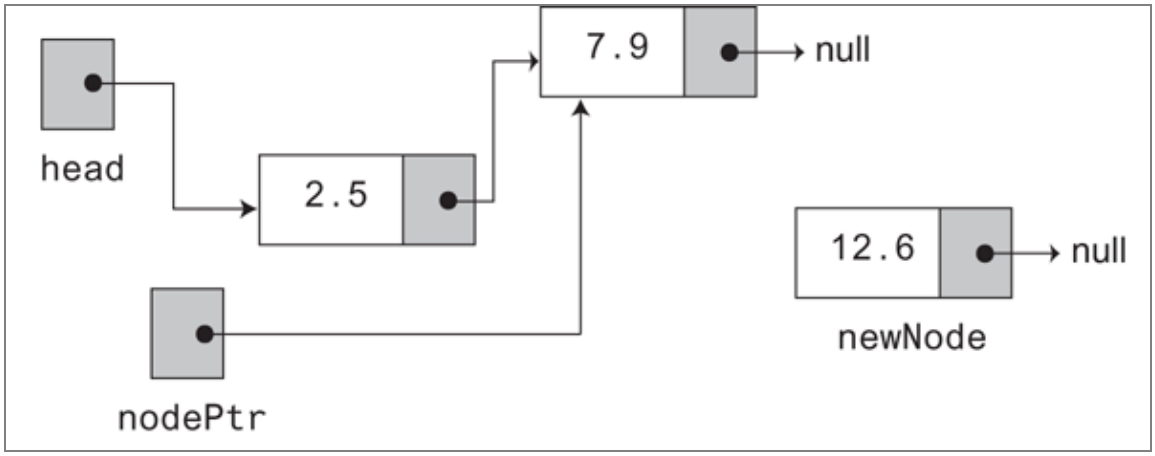


Figure 18-10 `nodePtr` points to the second node

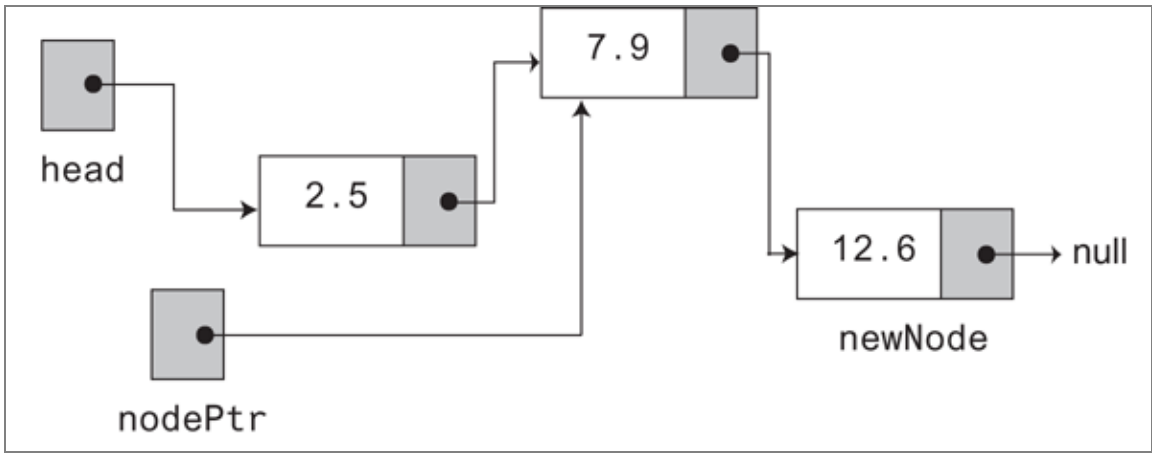


Figure 18-11 The new node is added to the list

- Finally, `destroyList` is called to deallocate all nodes.

Traversing a Linked List

Traversing a list means visiting each node in sequence, typically to perform an operation like displaying its data. The `displayList` function demonstrates this process.

The pseudocode for traversing is:

```
Assign list head to a node pointer.
While the node pointer is not null
    Process the data in the current node.
    Advance the node pointer to the next node.
End While.
```

The C++ implementation of `displayList`:

```
void displayList(const ListNode *head)
{
    ListNode *nodePtr;

    nodePtr = head;

    while (nodePtr)
    {
        std::cout << nodePtr->value << std::endl;

        nodePtr = nodePtr->next;
```

```
    }
}
```

Program 18-2 shows this function in action.

Program 18-2

```
#include <iostream>
#include "numberlist.h"

int main()
{
    ListNode *head = nullptr;

    appendNode(head, 2.5);
    appendNode(head, 7.9);
    appendNode(head, 12.6);

    displayList(head);

    destroyList(head);

    return 0;
}
```

Program Output

```

```

Traversal algorithms are fundamental to many linked list operations.

Inserting a Node

Inserting a Node in a Linked List

Inserting a node into a sorted list is more complex than appending. The goal is to add a new node while maintaining the list's sorted order.

The pseudocode for this algorithm is:

```
Create a new node and store data in it.
If the list is empty
    Make the new node the first node.
Else
    Find the correct insertion point.
    Insert the new node at that position.
End If.
```

The insertion logic requires two pointers: `nodePtr` to scan the list and `previousNode` to track the node before the insertion point.

The traversal part of the algorithm:

```
nodePtr = head;
previousNode = nullptr;
while (nodePtr != nullptr && nodePtr->value < num)
{
    previousNode = nodePtr;
    nodePtr = nodePtr->next;
}
```

The complete `insertNode` function:

```
void insertNode(ListNode *&head, double num)
{
    ListNode *newNode;
    ListNode *nodePtr;
    ListNode *previousNode = nullptr;

    newNode = new ListNode;
    newNode->value = num;

    if (!head)
    {
        head = newNode;
        newNode->next = nullptr;
    }
    else
    {
        nodePtr = head;
        previousNode = nullptr;

        while (nodePtr != nullptr && nodePtr->value < num)
```

```
        {
            previousNode = nodePtr;
            nodePtr = nodePtr->next;
        }

        if (previousNode == nullptr)
        {
            head = newNode;
            newNode->next = nodePtr;
        }
        else
        {
            previousNode->next = newNode;
            newNode->next = nodePtr;
        }
    }
}
```

Program 18-3 demonstrates inserting a node into an ordered list.

Program 18-3

```
#include <iostream>
#include "numberlist.h"

int main()
{
    ListNode *head = nullptr;

    appendNode(head, 2.5);
    appendNode(head, 7.9);
    appendNode(head, 12.6);

    insertNode(head, 10.5);

    displayList(head);

    destroyList(head);

    return 0;
}
```

Program Output

- **Step-by-step execution of `insertNode(head, 10.5)`:**
  - A new node with value 10.5 is created.
  - Since the list is not empty, the `else` block executes.
  - `nodePtr` starts at `head` (Figure 18-12).

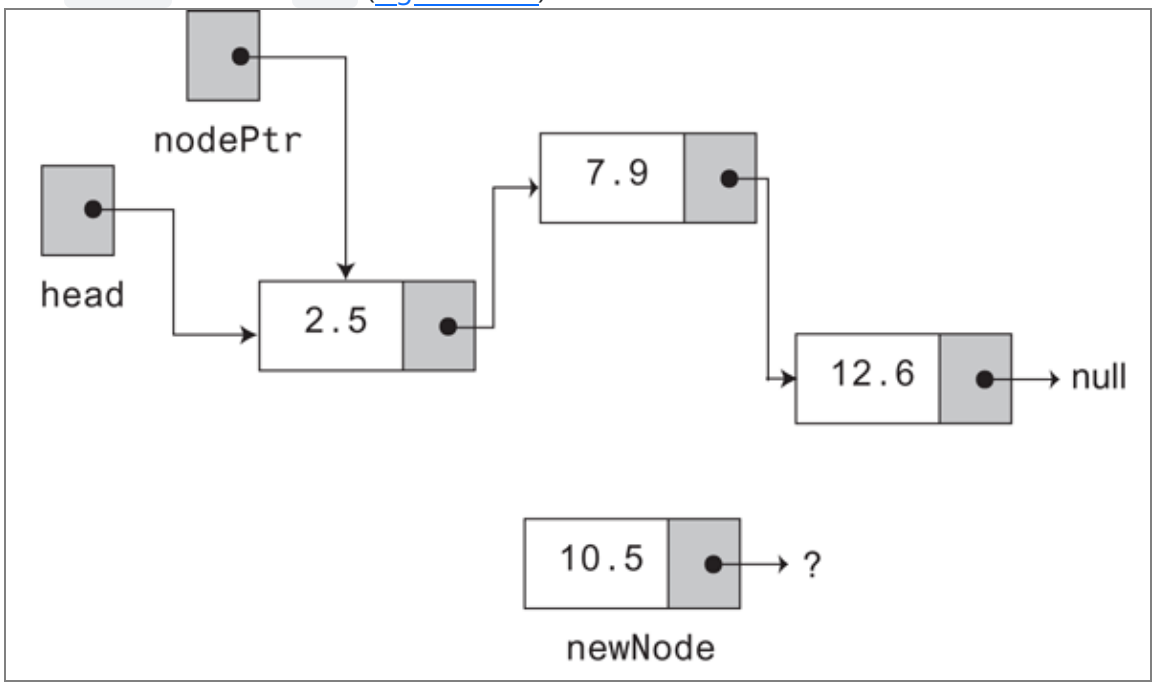


Figure 18-12 `nodePtr` points to the first node

  - The `while` loop iterates, advancing `previousNode` and `nodePtr` until `nodePtr` points to a node whose value is not less than 10.5 (the node with value 12.6) (Figure 18-13 and Figure 18-14).

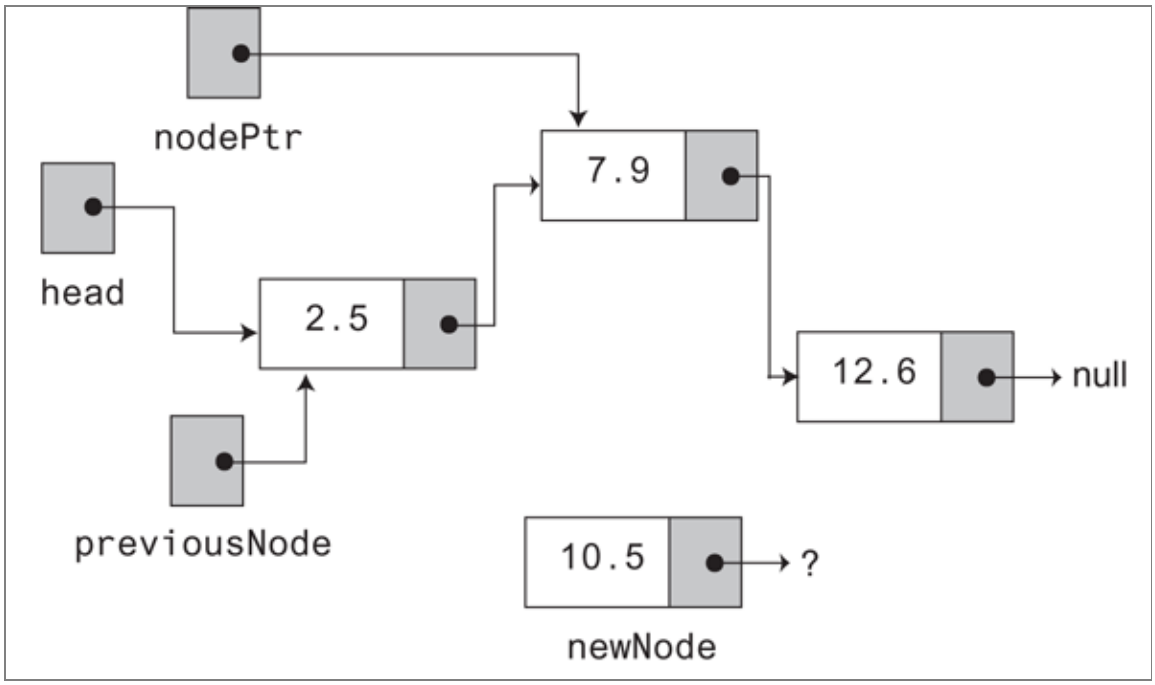


Figure 18-13 `previousNode` points to the first node and `nodePtr` points to the second node

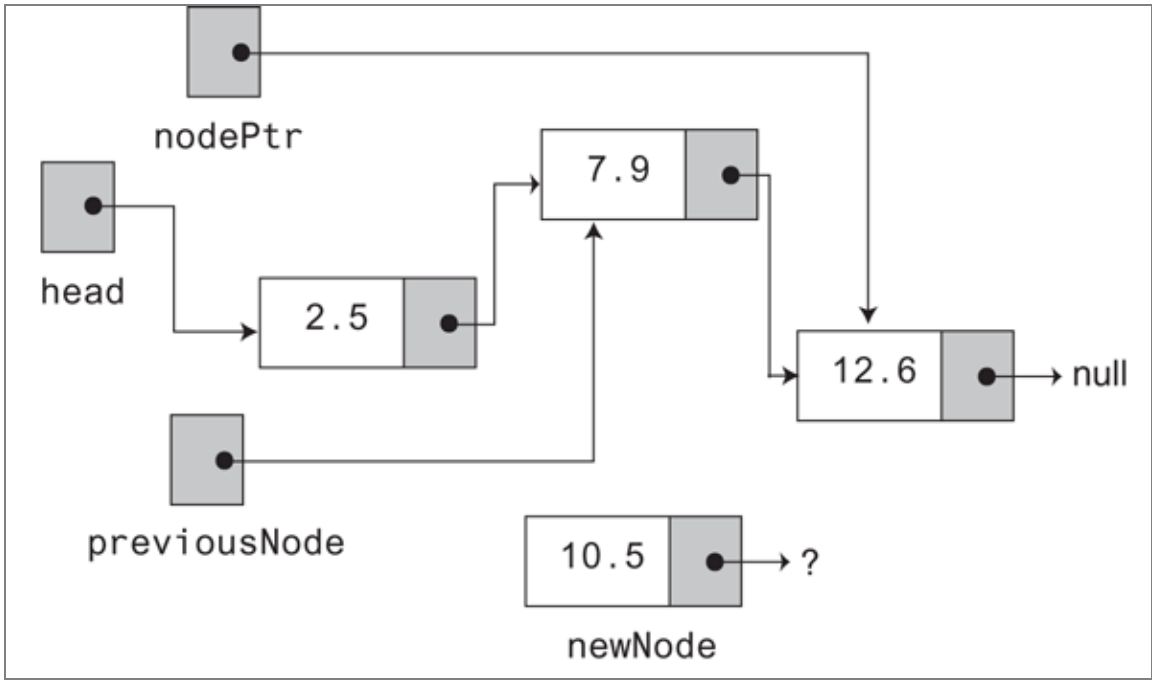


Figure 18-14 `previousNode` points to the second node and `nodePtr` points to the third node

- The loop terminates. `previousNode` points to the node with value 7.9.
- `previousNode->next` is set to point to `newNode`.
- `newNode->next` is set to point to `nodePtr`, inserting the new node between the nodes with values 7.9 and 12.6 (Figure 18-15).

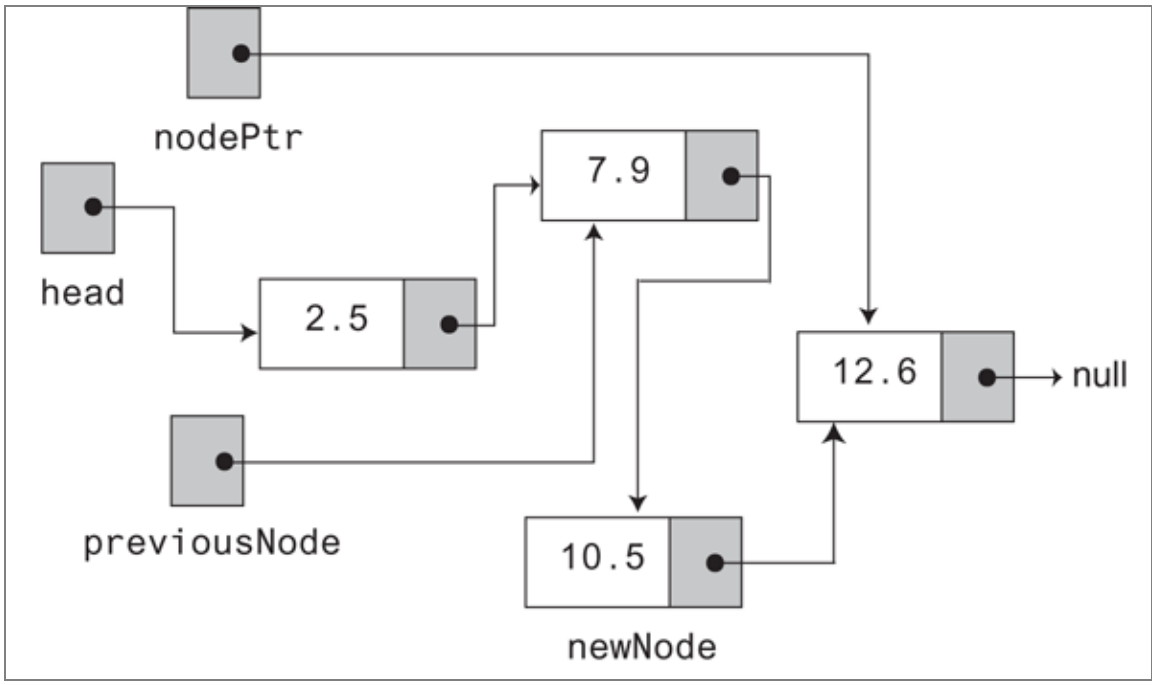


Figure 18-15 The new node inserted

Checkpoint

18.5 What is the difference between appending a node and inserting a node?

18.6 Which is generally easier to code, appending or inserting?

18.7 Why does the `insertNode` function use a `previousNode` pointer?

Deleting a Node

Deleting a Node from a Linked List

Deleting a node involves two steps:

1. Remove the node from the list by adjusting pointers to bypass it.
2. Free the memory allocated for the node using the `delete` operator.

The `deleteNode` function finds a node with a specific value and removes it. It uses `nodePtr` and `previousNode` to traverse the list. When `nodePtr` finds the node to be deleted, `previousNode->next` is redirected to `nodePtr->next`, effectively unlinking the target node. Finally, the memory for the unlinked node is deallocated.

```
void deleteNode(ListNode *&head, double num)
{
```

```
    ListNode *nodePtr;
    ListNode *previousNode;

    if (!head)
        return;

    if (head->value == num)
    {
        nodePtr = head->next;
        delete head;
        head = nodePtr;
    }
    else
    {
        nodePtr = head;

        while (nodePtr != nullptr && nodePtr->value != num)
        {
            previousNode = nodePtr;
            nodePtr = nodePtr->next;
        }

        if (nodePtr)
        {
            previousNode->next = nodePtr->next;
            delete nodePtr;
        }
    }
}
```

Program 18-4 demonstrates the `deleteNode` function.

Program 18-4

```
#include <iostream>
#include "numberlist.h"

int main()
{
    ListNode *head = nullptr;

    appendNode(head, 2.5);
    appendNode(head, 7.9);
    appendNode(head, 12.6);
```

```
    std::cout << "Here are the initial values:\n";
    displayList(head);
    std::cout << std::endl;

    std::cout << "Now deleting the node in the middle.\n";
    deleteNode(head, 7.9);

    std::cout << "Here are the nodes left.\n";
    displayList(head);
    std::cout << std::endl;

    std::cout << "Now deleting the last node.\n";
    deleteNode(head, 12.6);

    std::cout << "Here are the nodes left.\n";
    displayList(head);
    std::cout << std::endl;

    std::cout << "Now deleting the only remaining node.\n";
    deleteNode(head, 2.5);

    std::cout << "Here are the nodes left.\n";
    displayList(head);
    return 0;
}
```

Program Output

```

```

- **Walkthrough of deleting the node with value 7.9:**

- The function traverses the list until `nodePtr` points to the node with value 7.9 and `previousNode` points to the preceding node (with value 2.5) (Figure 18-16).

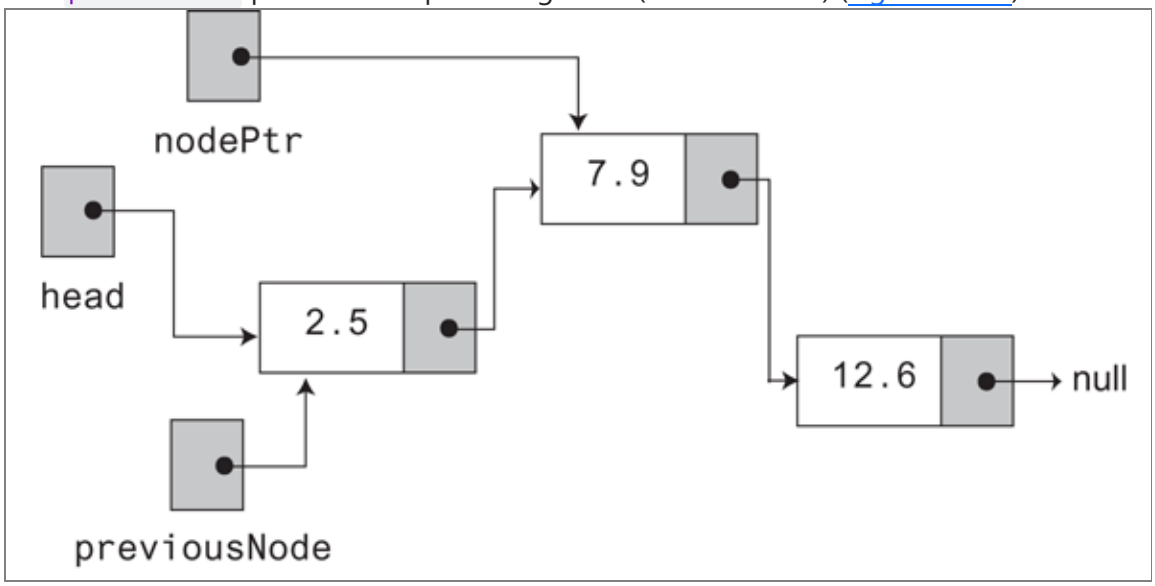


Figure 18-16 `previousNode` points to the first node and `nodePtr` points to the second node

- The statement `previousNode->next = nodePtr->next;` is executed. This redirects the `next` pointer of the first node to point to the third node, bypassing the second node (Figure 18-17).

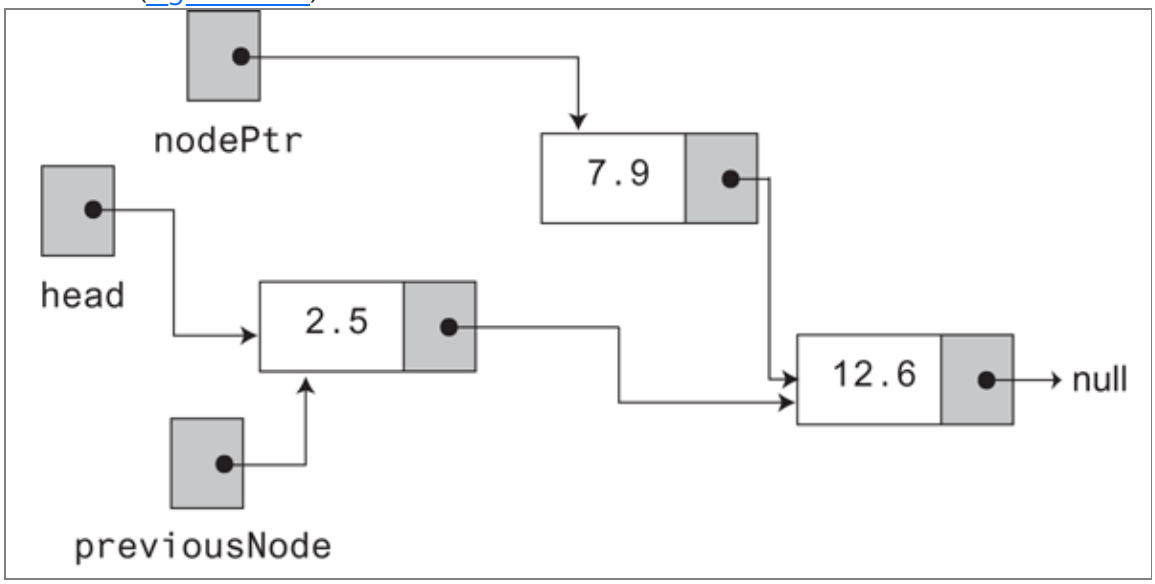


Figure 18-17 Node removed from the list

- The `delete nodePtr;` statement then frees the memory occupied by the unlinked node.

Destroying the List

To prevent memory leaks, all memory used by the list must be deallocated. The `destroyList` function traverses the list, deleting each node one by one.

```
void destroyList(ListNode *&head)
{
    ListNode *nodePtr;
    ListNode *nextNode;

    nodePtr = head;

    while (nodePtr != nullptr)
    {
        nextNode = nodePtr->next;

        delete nodePtr;

        nodePtr = nextNode;
    }
    head = nullptr;
}
```

A `nextNode` pointer is necessary to hold the address of the next node before the current node (`nodePtr`) is deleted. After destroying all nodes, `head` is set back to `nullptr`.

Checkpoint

18.8 What are the two steps involved in deleting a node from a linked list?

18.9 When deleting a node, why must you first unlink it before using the `delete` operator?

18.10 What might eventually happen in a large program that uses linked lists if a function like `destroyList` is not called to free the memory?